Database Module Script

Part 1: Intro

"Hi, we're Group 9 — the Database team for CSE 498, Company C. Our module is the shared storage backbone for the entire project: a Redis-style key-value storage system written in C++ that every other group calls directly.

The database supports two backends — a fast in-memory store for hot simulation state, and a SQLite-backed mode for persistence across sessions. To keep it simple and easy to use, regardless of the backend chosen callers can still use the Database exactly the same.

On top of that, we built a full serialization layer, LZW string compression, delta encoding on updates, and a WebSocket sync layer so game state can be saved and loaded across a network.

We'll walk through the four core pieces: Serialization, String Compression, WebSocket and SQL persistence, and then wrap up."

Part 2: Serialization

"The foundation of the database is the `Serializer` class. Any time you call `Store` or `Load`, the object gets converted to and from a raw byte string under the hood.

For built-in types — `int`, `double`, `bool`, `std::string`, vectors, maps — this happens automatically. But we also support custom game objects through `RegisterType`. You give it a name, a lambda to serialize your type to a string, and a lambda to deserialize it back. After that, `Store` and `Load` work on your type exactly like any built-in.

The three shared project types — `DataGrid`, `WorldPosition`, and `Location` — are pre-registered inside the `Database` constructor, so other groups don't have to do anything for those.

The serialized bytes also carry an explicit format byte — `Raw`, `Compressed`, or `DiffCompressed` — so the database always knows how to decode what it reads back. On `Update`, if a diff against the existing value is smaller than a full replacement, we store a diff-backed entry instead. `StringDiff` computes the patch; on load, it applies the patch against the base to reconstruct the value."

Part 3: String Compression

"Once a value is serialized to a string, it goes through our compression layer before being stored. We implemented LZW compression in StringCompressor. When auto_compress is enabled in the config and a serialized value exceeds the threshold — 100 bytes by default — it gets compressed before storage. The format byte we just mentioned gets set to Compressed, and on load, the database decompresses transparently. The caller never sees any of this.

LZW is a dictionary-based algorithm. It scans the input and builds a table of repeated substrings, replacing them with shorter numeric codes. The longer the input and the more repetition it contains, the better it compresses. For repetitive game data — world chunk states, repeated field names, grid patterns — it achieves meaningful size reduction.

We chose LZW because it's a single-pass algorithm with no dependencies. The compressor exposes a GetCompressionRatio method so you can measure exactly how much a given value shrank.

The second half of this layer is StringDiff. When you call Update on a key that already exists, rather than storing a full replacement, we compute a patch from the old value to the new one using common prefix and suffix heuristics. If the patch is smaller than a full snapshot — which it usually is for incremental simulation updates — we store the diff instead, and the format byte is set to DiffCompressed."

Part 4: WebSocket Connection & SQL Persistence

> "The database has two persistence layers.
>
> The first is SQLite. Pass a file path to the constructor and the database switches from in-memory `unordered_map` to SQLite-backed storage. All reads and writes go through `SQLiteConnection`, which wraps the SQLite C API. Data survives process restarts, and the same file can be reopened by a new `Database` instance and all entries are still there. WAL mode is on by default for better concurrent read performance.
>
> The second layer is the WebSocket sync stack. `WebSocketServer` and `WebSocketConnection` wrap IXWebSocket, and `SyncManager` sits on top of both. The protocol is simple: each frame is a message type byte followed by an 8-byte big-endian length and then a payload.
>
> The save/load flow works like this: a client calls `SaveGame` with a name, the `SyncManager` serializes the full database state into a payload and sends a `SAVE` frame to the server, and the server writes it to disk as a `.save` file. To restore, the

client sends a `LOAD` frame with the save name, the server reads the file and sends back the state, and the client's database is repopulated. The client can also call `RequestSaveList` to get a timestamped list of available saves.

`SendUpdate` lets a client push individual key changes to the server in real time using delta frames, so the server's state stays in sync tick by tick without sending full snapshots."

Part 5: Outro

To summarize, the system we've built is structured around two layers: a networking layer and a data management layer, each designed to be modular, efficient, and extensible.

On the networking side, the WebSocket components provide a clean abstraction for real-time communication. The client-side connection class encapsulates a single connection with an event-driven interface, while the server manages multiple clients using unique IDs and exposes callbacks for connection, disconnection, and messaging.

On the data side, the Database class acts as a high-level key-value store, but with a data pipeline that supports custom type serialization, optional compression, and differential storage to minimize the amount of data being written or transmitted. It can operate purely in memory or persist data using SQLite, and it includes transaction support for safe updates. Importantly, it also tracks changes to data over time, which allows us to efficiently identify what needs to be synchronized.

The API is intentionally minimal — `Store`, `Load`, `Update`, `Delete`, `FindKeys`. Every method returns `std::expected` so callers handle errors explicitly without exceptions. Other groups include one header and they're ready to go.

With all of this, you get a database designed for real-time state synchronization. Changes in the database can be tracked, batched, and sent over the network using the WebSocket server, allowing connected clients to stay in sync with minimal overhead.

Thank you for watching!